

Code-Layout, Readability and Reusability

Date	29 September 2026 – 2 October 2026
Team ID	6ab4fddea238dc999eb665e9
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the PocketSmart AI codebase in terms of its structure, readability, and reusability. The project uses a modular FastAPI structure with separate configuration, authentication, database, schema, AI utility, templates, static assets and tests.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent Python indentation and formatting.
2	Proper File Structure	Files and folders are logically organized	Yes	FastAPI modules, templates, static files and tests are separated by purpose.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as budget, preferences, user_id and recommendations are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Authentication, planning, recommendation and history functions use descriptive names.
5	Code Comments	Inline and block comments explain logic	Yes	Important configuration, fallback and integration logic is documented.
6	Modular Design	Code is split into reusable functions/modules	Yes	Authentication, database, schemas, configuration and Gemini utilities are modular.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared utilities avoid duplicating common planner and authentication logic.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Validation, upload, authentication and Gemini failures have controlled handling.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Authentication (auth.py)	Python / FastAPI / JWT	Registration, login, logout and protected flows	High
2	Gemini utility (gemini_utils.py)	Python / Gemini API	Home, Party and Jewelry recommendations	High
3	Configuration (config.py)	Python / environment variables	API keys, model and app configuration	High
4	Database / models	Python / SQLite	Users, sessions, recommendations and history	High
5	Schemas / validation	Python / Pydantic	Planner inputs and structured AI output	High

C O D E

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
2	Gemini utility (gemini_utils.py)	Python / Gemini API	Home, Party and Jewelry recommendations	High
3	Configuration (config.py)	Python / environment variables	API keys, model and app configuration	High
4	Database / models	Python / SQLite	Users, sessions, recommendations and history	High
5	Schemas / validation	Python / Pydantic	Planner inputs and structured AI output	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear FastAPI module structure with separated templates, static assets and tests.
Readability	4	Descriptive naming and separation of responsibilities make the project readable.
Reusability	5	Shared authentication, configuration, database, schema and Gemini utilities serve multiple planners.
Documentation / Comments	4	README and modular structure document the project; focused comments support complex integration logic.
Overall Score	4.5	Strong modular organization and reuse for the PocketSmart AI prototype.